

DocuMind AI

Technical Documentation & Project Reference Guide

Project: DocuMind AI — Intelligent Document Analysis & Agent Platform

Primary Goal: Build a practical GenAI document intelligence platform combining document processing, OCR, RAG, structured extraction, tool-calling agents, and multi-document comparison.

Status: Core functionality completed and tested successfully through the FastAPI APIs and Streamlit application.

Table of Contents

1. [Project Overview & Objectives](#)
 2. [Core Features](#)
 3. [Technology Stack](#)
 4. [System Architecture](#)
 5. [End-to-End Workflow](#)
 6. [Project Directory Structure](#)
 7. [Complete File Reference](#)
 8. [Configuration & Environment Variables](#)
 9. [Installation & Setup](#)
 10. [Running the Application](#)
 11. [API Reference](#)
 12. [RAG Architecture](#)
 13. [Structured Extraction](#)
 14. [Tool-Calling Agent](#)
 15. [Multi-Document Comparison](#)
 16. [Testing](#)
 17. [Evaluation Framework](#)
 18. [Important Design Decisions](#)
 19. [Edge Cases & Error Handling](#)
 20. [Gemini Migration](#)
 21. [Known Limitations](#)
 22. [Future Improvements](#)
 23. [Project Revision Checklist](#)
 24. [Resume-Level Project Summary](#)
-

1. Project Overview & Objectives

1.1 What is DocuMind AI?

DocuMind AI is an intelligent document analysis platform designed to allow users to upload documents and interact with their contents using modern GenAI techniques.

Instead of treating a document simply as a collection of text, the system provides multiple ways of interacting with it:

- Natural-language question answering
- Retrieval-Augmented Generation
- Source/page-aware citations
- Structured information extraction
- Tool-calling document agents
- Multi-document comparison
- Inconsistency/mismatch detection
- OCR-based document processing

The project was intentionally designed to be more than a basic **"Chat with PDF"** application.

1.2 Problem Being Solved

Traditional document workflows often require users to:

1. Open large PDFs manually.
2. Search for keywords.
3. Read multiple pages to understand context.
4. Compare different documents manually.
5. Extract important fields into structured formats.
6. Verify where an answer originated.

This becomes difficult with:

- Contracts
- Agreements
- Invoices
- Forms
- Legal documents
- Scanned documents
- Multiple related documents

DocuMind AI attempts to reduce this manual effort by combining document processing with LLM-based reasoning and retrieval.

1.3 Project Objective

The primary objective was to build a **practical, portfolio-quality GenAI system** demonstrating knowledge of:

- Python
- LangChain
- RAG
- Embeddings
- Vector databases/search
- LLM integration

- Structured output
 - Tool calling
 - Agents
 - OCR
 - FastAPI
 - Streamlit
 - Pydantic
 - Testing
 - Evaluation
-

2. Core Features

2.1 Document Ingestion

Documents can be uploaded through the API/frontend and processed into an internal representation.

The system generates a unique document ID and maintains page-level information.

2.2 PDF Processing

PDF documents are processed using PyMuPDF.

Text is extracted page-by-page rather than treating the entire document as one large string.

This is important because page information is later used for citations.

2.3 OCR

Images and documents without usable text can be processed using:

- Pillow
- pytesseract
- Tesseract OCR

OCR allows scanned/image-based documents to participate in the same RAG workflow.

2.4 Preprocessing

Extracted text goes through conservative preprocessing before being chunked.

The objective is to remove obvious noise while preserving useful document structure.

2.5 Page-Aware Chunking

Documents are split into LangChain `Document` chunks.

Each chunk retains metadata such as:

- Document ID
- Filename
- Page number
- Document metadata

This allows retrieved context to remain traceable to the original document.

2.6 Gemini Embeddings

The final project uses Google's Gemini embedding infrastructure.

The embedding model discovered and successfully used during implementation was:

```
gemini-embedding-001
```

The earlier configuration:

```
text-embedding-004
```

failed because it was unavailable for the API configuration being used.

2.7 FAISS Vector Search

FAISS is used as the local vector store.

Document chunks are embedded and stored for similarity retrieval.

2.8 Retrieval-Augmented Generation

When a user asks a question:

```
Question
↓
Embedding
↓
FAISS similarity search
↓
```

```
Relevant chunks
↓
Context construction
↓
Gemini LLM
↓
Grounded answer
```

2.9 Source/Page Citations

The RAG pipeline preserves document and page metadata.

The generated context contains page/document markers so that answers can provide source information.

Example conceptual context:

```
[Document: Affiliate Agreement.pdf | Page: 3]
...
```

This helps users understand where information originated.

2.10 Structured Information Extraction

The extraction service converts unstructured document information into structured Pydantic-based output.

The result includes extracted fields and supporting metadata such as confidence/source information where applicable.

2.11 Tool-Calling Agent

The agent provides a higher-level interface where an LLM can use document-related tools.

For example:

```
Find the effective date of this agreement and identify the parties involved.
```

The agent can determine which document capability is appropriate instead of relying exclusively on a direct RAG call.

2.12 Multi-Document Comparison

The system can compare two documents.

The comparison workflow extracts relevant fields, normalizes values, compares them, and identifies differences.

This is useful for:

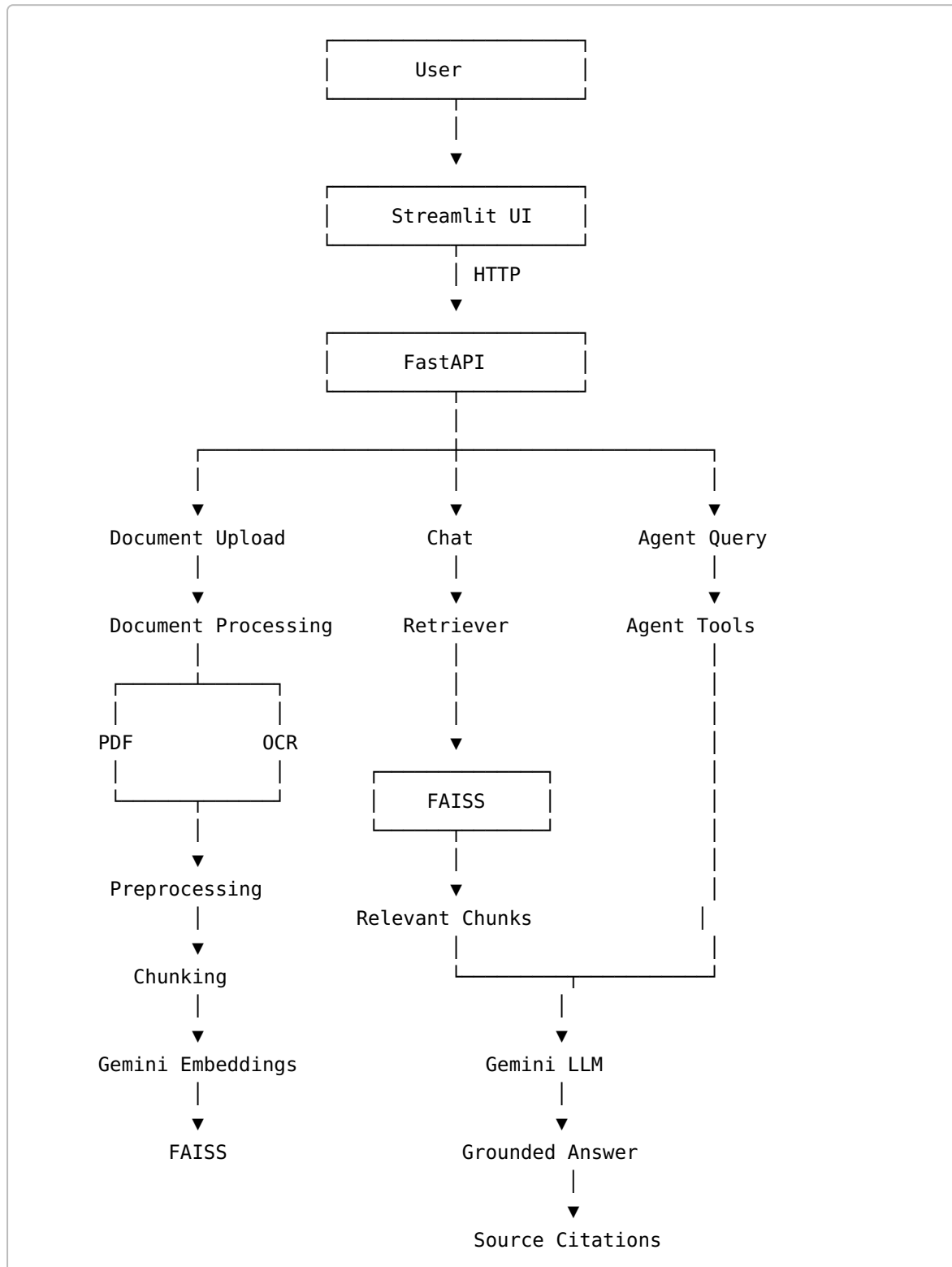
- Contract versions
- Agreements
- Invoices
- Related legal/business documents

3. Technology Stack

Technology	Purpose
Python	Core programming language
LangChain	LLM, RAG and agent orchestration
Gemini	LLM and embeddings
FAISS	Local vector similarity search
FastAPI	REST API
Streamlit	User interface
Pydantic	Data validation and schemas
PyMuPDF	PDF processing
Pillow	Image handling
pytesseract	OCR integration
Tesseract	OCR engine
pytest	Automated testing
python-dotenv / Pydantic Settings	Configuration
Docker	Containerization when configured

4. System Architecture

4.1 High-Level Architecture



5. End-to-End Workflow

5.1 Document Upload Workflow

The upload process is:

```
User uploads PDF/image
↓
FastAPI /documents/upload
↓
Document Loader
↓
PDF extraction / image loading
↓
OCR if necessary
↓
Preprocessing
↓
Page-aware chunking
↓
Gemini embeddings
↓
FAISS vector store
↓
Document indexed successfully
```

5.2 Question Answering Workflow

```
User question
↓
POST /chat
↓
RAG Pipeline
↓
Question embedding
↓
FAISS retrieval
↓
Top relevant chunks
↓
Context construction
↓
Grounding prompt
↓
Gemini LLM
↓
```

Answer
↓
Source/page citations

5.3 Extraction Workflow

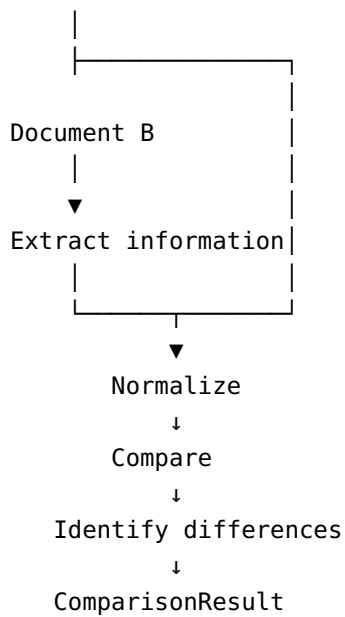
Document
↓
Extraction Service
↓
Document context
↓
Gemini structured output
↓
Pydantic validation
↓
StructuredExtractionResult
↓
Extracted fields + confidence/source

5.4 Agent Workflow

User query
↓
Document Agent
↓
Gemini tool-calling
↓
Select document tool
↓
Tool execution
↓
Retrieved/extracted information
↓
Gemini final response

5.5 Comparison Workflow

Document A
|
▼
Extract comparable information



6. Project Directory Structure

The project was organized approximately as follows:

```
documind-ai/
├── app/
│   ├── api/
│   │   └── main.py
│   ├── document/
│   │   ├── loader.py
│   │   ├── ocr.py
│   │   └── preprocessing.py
│   ├── rag/
│   │   ├── chunker.py
│   │   ├── embeddings.py
│   │   ├── vector_store.py
│   │   ├── retriever.py
│   │   └── pipeline.py
│   ├── agents/
│   │   ├── tools.py
│   │   └── document_agent.py
│   └── services/
│       ├── extraction_service.py
│       └── comparison_service.py
```

```

|
|
|   └─ models/
|       └─ schemas.py
|
|   └─ core/
|       ├── config.py
|       └─ logger.py
|
└─ frontend/
    └─ app.py

└─ tests/
    ├── test_api.py
    ├── test_rag.py
    ├── test_services.py
    ├── test_document.py
    ├── test_agent.py
    └─ test_evaluator.py

└─ evaluation/
    ├── questions.json
    └─ evaluator.py

└─ scripts/

└─ data/
    ├── raw/
    │   ├── invoices/
    │   ├── forms/
    │   ├── contracts/
    │   └─ synthetic/
    │
    ├── processed/
    └─ vector_store/

└─ requirements.txt
└─ .env
└─ .gitignore
└─ README.md

```

Note: The tree above reflects the project files and directories created/reviewed during our implementation. Any automatically generated Python cache files such as `__pycache__` are intentionally excluded because they are runtime artifacts rather than project source files.

7. Complete File Reference

7.1 `app/core/config.py`

Primary Purpose

Central configuration management for the application.

The configuration was designed to avoid scattering environment-variable access throughout the project.

It became especially important during the migration from OpenAI to Gemini.

Key Components

The module contains the application settings object and cached settings access.

Configuration covers areas such as:

- Project information
- API host/port
- LLM provider
- LLM model
- LLM temperature
- Embedding provider
- Embedding model
- Gemini API key
- Chunk size
- Chunk overlap
- Retrieval `top_k`
- Data directories

Dependencies

Uses Pydantic/Pydantic Settings and environment configuration.

Interactions

Used by:

- `app/rag/embeddings.py`
- `app/rag/chunker.py`
- `app/rag/pipeline.py`
- `app/services/extraction_service.py`
- `app/agents/document_agent.py`
- API/application configuration

Importance

This is the central configuration layer.

A major design decision was to prevent API keys and model configuration from being hardcoded inside individual modules.

7.2 `app/core/logger.py`

Primary Purpose

Provides centralized application logging.

Key Component

A `get_logger` utility creates consistently configured loggers.

Dependencies

Python logging facilities.

Interactions

Used throughout the application modules.

Importance

Provides consistent logging and makes debugging easier without duplicating logging configuration in every file.

7.3 `app/models/schemas.py`

Primary Purpose

Defines shared Pydantic data models.

Important Models

The project contains schemas including:

- `DocumentMetadata`
- `PageContent`
- `ProcessedDocument`
- `ExtractedField`
- `SourceCitation`
- `ChatRequest`
- `ChatResponse`
- `ComparisonField`
- `ComparisonResult`

Dependencies

Pydantic.

Interactions

Used by:

- Document processing
- RAG
- Extraction
- Comparison
- Agent
- API routes
- Tests

Importance

Provides a consistent contract between different parts of the application.

7.4 `app/document/loader.py`

Primary Purpose

Loads documents and converts them into the project's internal document representation.

Main Responsibilities

- PDF loading
- Page extraction
- Image loading
- Document ID generation
- Metadata generation
- Page-level content representation
- Full-text assembly

Key Internal Logic

The PDF loader processes pages individually.

Each page becomes a `PageContent` object containing text and page information.

Dependencies

- PyMuPDF
- Pillow
- Project schemas
- Logging

Interactions

Feeds into:

```
preprocessing.py
  ↓
chunker.py
```

Importance

This is the entry point for raw document data.

7.5 `app/document/ocr.py`

Primary Purpose

Provides OCR functionality for image/scanned content.

Main Responsibilities

- Convert images into text
- Interface with Tesseract
- Handle OCR errors
- Provide OCR output to document processing

Dependencies

- Pillow
- pytesseract
- Tesseract installation

Interactions

Works with document loading/processing.

Importance

Allows DocuMind AI to work with documents that don't contain machine-readable text.

7.6 `app/document/preprocessing.py`

Primary Purpose

Cleans extracted document text before indexing.

Main Responsibilities

- Remove unnecessary noise
- Normalize text conservatively
- Preserve meaningful line/document structure
- Process page-level content

Design Decision

The preprocessing layer was intentionally conservative.

Aggressive whitespace normalization can damage tables and document structure, so the system avoids destroying meaningful formatting.

Dependencies

Project document schemas and logging.

Interactions

```
loader.py / OCR
  ↓
preprocessing.py
  ↓
chunker.py
```

Importance

Provides cleaner input to the chunking and retrieval layers.

7.7 app/rag/chunker.py

Primary Purpose

Splits processed documents into retrieval-ready LangChain documents.

Main Responsibilities

- Page-by-page processing
- Recursive text splitting
- Chunk size configuration
- Chunk overlap
- Metadata preservation

Important Design Decision

Chunking is performed **page-by-page**.

This prevents page information from being lost and makes source citations much easier.

Dependencies

- LangChain document types
- RecursiveCharacterTextSplitter
- Application settings
- Project schemas

Interactions

Receives:

ProcessedDocument

and produces:

list[Document]

with metadata.

Importance

This is the bridge between document processing and vector retrieval.

7.8 app/rag/embeddings.py

Primary Purpose

Provides the application's embedding service.

Current Provider

Gemini.

Current Embedding Model

gemini-embedding-001

Main Responsibilities

- Validate embedding configuration
- Initialize Gemini embedding model
- Embed documents
- Embed user queries

- Handle provider/API errors
- Provide vectors to FAISS

Important Migration History

The original implementation used OpenAI embeddings.

Because OpenAI API billing/card setup was unavailable, the project was migrated to Gemini.

The migration removed the dependency on OpenAI embeddings from the active runtime path.

Important Debugging Issue

The initial Gemini configuration attempted:

```
text-embedding-004
```

The Gemini API returned:

```
404 NOT_FOUND
models/text-embedding-004 is not found
```

The available embedding models were then queried from the API:

```
models/gemini-embedding-001
models/gemini-embedding-2-preview
models/gemini-embedding-2
```

The project was configured with:

```
gemini-embedding-001
```

and document upload subsequently succeeded.

Importance

This module is critical because every indexed document requires embeddings before it can be searched through FAISS.

7.9 app/rag/vector_store.py

Primary Purpose

Manages the persistent FAISS vector store.

Main Responsibilities

- Create FAISS index
- Load existing index
- Add documents
- Rebuild vector store
- Persist vector store
- Return configured FAISS store

Important Security Decision

The implementation uses:

```
allow_dangerous_deserialization=True
```

when loading the local FAISS store.

This is acceptable only when the persisted vector store is considered trusted/local.

It should not be blindly enabled for arbitrary untrusted files.

Dependencies

- FAISS
- LangChain community vector store integration
- Embedding service
- Application configuration

Interactions

```
chunker.py
  ↓
embeddings.py
  ↓
vector_store.py
```

Importance

Provides persistent vector search storage.

7.10 app/rag/retriever.py

Primary Purpose

Retrieves the most relevant document chunks from FAISS.

Main Responsibilities

- Similarity retrieval
- Top-k configuration
- Query validation
- Document filtering
- Retrieval with scores

Important Detail

FAISS L2 scores represent **distance**, not confidence.

Therefore:

```
lower distance = more similar  
higher distance = less similar
```

They should not be presented as percentages without proper calibration.

Filtering

The retriever can filter based on document IDs.

The implementation retrieves a larger candidate pool when filtering before selecting the requested top-k.

Importance

This is the core search layer for RAG.

7.11 app/rag/pipeline.py

Primary Purpose

Implements the complete RAG question-answering pipeline.

Main Responsibilities

1. Receive question
2. Retrieve relevant chunks
3. Build context
4. Preserve page/document metadata
5. Apply grounding instructions
6. Invoke Gemini
7. Construct citations
8. Deduplicate citations

Grounding

The prompt is designed to encourage answers based on retrieved document context instead of unsupported information.

Context Format

Retrieved content includes markers conceptually similar to:

```
[Document: filename | Page: N]
```

Importance

This is the primary implementation of document question answering.

7.12 app/agents/tools.py

Primary Purpose

Defines the tools available to the document agent.

Main Tools

The implementation includes document-oriented capabilities such as:

- Document question answering
- Document field extraction

Important Helper Logic

The module includes logic for:

- Parsing document text
- Interpreting [Page N] markers
- Building ProcessedDocument instances
- Connecting tools with the RAG and extraction services

Dependencies

- LangChain tools
- RAG pipeline
- Extraction service
- Project schemas

Interactions

```
document_agent.py
  ↓
tools.py
  ├── RAGPipeline
  └── ExtractionService
```

Importance

Provides controlled capabilities that the LLM agent can invoke.

7.13 app/agents/document_agent.py

Primary Purpose

Implements the tool-calling document agent.

Main Responsibilities

- Initialize Gemini LLM
- Register document tools
- Construct agent prompts
- Execute tool-calling workflow
- Limit agent iterations
- Return final response

Agent Concept

Instead of manually deciding:

```
"I should call RAG"
```

the agent receives a user request and has access to defined document tools.

For example:

```
Find the effective date and identify the parties.
```

The agent can use the available document capabilities.

Importance

Demonstrates agentic GenAI functionality beyond basic RAG.

7.14 app/services/extraction_service.py

Primary Purpose

Provides structured document information extraction.

Main Responsibilities

- Build extraction context
- Call Gemini
- Request structured output
- Validate extracted fields
- Preserve source information
- Return structured extraction results

Output

Uses project Pydantic schemas including extracted fields and confidence/source metadata.

Important Validation

Confidence values should remain within:

```
0.0 <= confidence <= 1.0
```

Importance

Transforms unstructured documents into structured application data.

7.15 app/services/comparison_service.py

Primary Purpose

Compares information extracted from two documents.

Main Responsibilities

- Extract comparable document information
- Normalize values
- Apply synonym mappings
- Compare values
- Detect mismatches
- Build comparison result
- Generate mismatch summaries

Important Logic

The service contains a synonym mapping mechanism.

Normalization allows semantically similar values to be compared more effectively.

Known Consideration

Generic numeric normalization can potentially make identifiers such as:

00123

and:

123

appear equivalent even when leading zeros are meaningful.

This should be considered if the system is extended to production use.

Importance

Provides one of the major differentiating features beyond ordinary RAG.

7.16 app/api/main.py

Primary Purpose

Defines the FastAPI application and API endpoints.

Main Routes

The application exposes routes including:

```
GET /health
POST /documents/upload
POST /chat
POST /documents/extract
POST /documents/compare
POST /agent/query
```

/health

Used to confirm the API is running.

`/documents/upload`

Responsible for:

1. Receiving uploaded file
2. Saving/processing file
3. Loading document
4. Preprocessing
5. Chunking
6. Generating embeddings
7. Indexing in FAISS

`/chat`

Runs RAG question answering.

`/documents/extract`

Runs structured extraction.

`/documents/compare`

Runs multi-document comparison.

`/agent/query`

Runs the document tool-calling agent.

CORS

The application was configured for local Streamlit communication.

Importance

Acts as the application backend and public interface between the frontend and internal services.

7.17 `frontend/app.py`

Primary Purpose

Implements the Streamlit user interface.

Major UI Areas

The frontend provides interfaces for:

- Upload & Index
- Ask Documents
- Extract

- Compare
- Agent

Communication

The frontend communicates with FastAPI using HTTP requests.

Conceptually:

```
Streamlit
  ↓ HTTP
FastAPI
  ↓
Application services
```

Importance

Provides the actual user-facing application and was used to validate the complete project workflow.

7.18 tests/test_api.py

Primary Purpose

Tests FastAPI endpoints.

Coverage

Includes testing around:

- Health endpoint
- Upload
- Chat
- Extraction
- Comparison
- Agent

Importance

Ensures the API contract remains stable.

7.19 tests/test_rag.py

Primary Purpose

Tests the RAG subsystem.

Areas Covered

- Chunking
- Embedding service
- Vector store
- Retriever
- RAG pipeline

Important Considerations

The tests were designed to avoid unnecessary dependence on live external APIs where possible.

Some mocking/monkeypatching areas are intentionally isolated.

Importance

Protects the project's most important GenAI pipeline.

7.20 tests/test_services.py

Primary Purpose

Tests business/service-layer functionality.

Coverage

Includes:

- ExtractionService
- ComparisonService

Importance

Ensures structured extraction and comparison logic work independently from the API layer.

7.21 tests/test_document.py

Primary Purpose

Tests document processing.

Coverage

Includes:

- Document loading
- PDF handling

- OCR-related behavior
- Preprocessing

Importance

Protects the ingestion layer.

7.22 tests/test_agent.py

Primary Purpose

Tests the document agent and tools.

Coverage

Includes:

- Agent initialization
- Tool behavior
- Document context handling
- Agent execution behavior

Importance

Protects the agentic layer.

7.23 evaluation/questions.json

Primary Purpose

Contains evaluation questions for measuring DocuMind's behavior.

Categories

The evaluation set includes categories such as:

- RAG
- Citation
- Extraction
- Reasoning
- Missing information
- Comparison
- Cross-document questions

Importance

Provides a repeatable evaluation dataset rather than relying exclusively on manual testing.

7.24 `evaluation/evaluator.py`

Primary Purpose

Implements deterministic evaluation logic.

Main Responsibilities

- Load evaluation questions
- Execute an answer function
- Evaluate responses
- Evaluate citation behavior
- Calculate summary scores
- Save evaluation reports

Citation Evaluation

An important correction was made to ensure citation scoring considers all questions where citations are required rather than accidentally excluding questions.

Importance

Allows the project to measure quality systematically.

7.25 `tests/test_evaluator.py`

Primary Purpose

Tests the evaluation framework itself.

Coverage

Includes:

- Evaluation scoring
- Question handling
- Citation scoring
- Report generation
- Missing-information handling

Importance

Ensures the evaluation system itself is reliable.

7.26 requirements.txt

Primary Purpose

Defines Python dependencies required by the application.

Main Dependency Groups

```
FastAPI ecosystem
LangChain ecosystem
Gemini integration
FAISS
PDF/OCR processing
Streamlit
Pydantic
Testing
```

Importance

Allows the project environment to be recreated.

7.27 .env

Primary Purpose

Stores local environment configuration and secrets.

Important Rule

This file must **never be committed to Git**.

The Gemini API key belongs here.

Example:

```
GEMINI_API_KEY=your_key
```

Actual model/provider configuration should match `app/core/config.py`.

7.28 `.gitignore`

Primary Purpose

Prevents sensitive and generated files from being committed.

Important entries should include:

```
.env  
.venv/  
__pycache__/  
*.pyc
```

and generated/local data where appropriate.

Importance

Protects API keys and keeps the Git repository clean.

7.29 `scripts/`

Primary Purpose

Contains project utility scripts.

The directory was reserved for operational/helper scripts rather than core application logic.

7.30 `data/raw/`

Primary Purpose

Stores original input documents.

The project dataset was organized into categories such as:

```
invoices/  
forms/  
contracts/  
synthetic/
```

7.31 data/processed/

Primary Purpose

Stores processed document artifacts when applicable.

7.32 data/vector_store/

Primary Purpose

Stores the persistent FAISS vector index.

This directory contains generated vector-store data rather than source code.

8. Configuration & Environment Variables

The final application is configured around Gemini.

The important configuration concept is:

```
GEMINI_API_KEY=your_gemini_api_key
```

The project also supports centralized settings for:

```
LLM_PROVIDER  
LLM_MODEL  
LLM_TEMPERATURE  
EMBEDDING_PROVIDER  
EMBEDDING_MODEL  
CHUNK_SIZE  
CHUNK_OVERLAP  
TOP_K  
API_HOST  
API_PORT
```

The exact configured values should always be taken from the project's current `.env` /configuration rather than hardcoded in documentation.

9. Installation & Setup

9.1 Create Virtual Environment

Windows

```
python -m venv .env
```

Activate:

```
.env\Scripts\activate
```

Linux/macOS

```
python3 -m venv .env  
source .env/bin/activate
```

9.2 Install Dependencies

```
pip install -r requirements.txt
```

9.3 Configure Gemini

Create `.env` in the project root.

Example:

```
GEMINI_API_KEY=your_gemini_api_key
```

Do not commit the real key.

9.4 OCR Setup

`pytesseract` is a Python interface.

The actual Tesseract OCR executable must also be installed on the machine.

If required, configure its executable path through the application's OCR configuration/environment variable.

10. Running the Application

10.1 Start FastAPI

From the project root:

```
uvicorn app.api.main:app --reload
```

Expected local API:

```
http://127.0.0.1:8000
```

10.2 Swagger Documentation

Open:

```
http://127.0.0.1:8000/docs
```

This provides interactive API testing.

10.3 Start Streamlit

In another terminal:

```
streamlit run frontend/app.py
```

The Streamlit UI can then be used to test:

- Upload
 - Q&A
 - Extraction
 - Comparison
 - Agent
-

11. API Reference

11.1 Health

GET /health

Purpose:

Confirm that the FastAPI application is running.

11.2 Upload Document

POST /documents/upload

Purpose:

Upload, process, embed and index a document.

Workflow:

Upload
→ Loader
→ OCR/processing
→ Chunking
→ Gemini embeddings
→ FAISS

11.3 Chat

POST /chat

Purpose:

Ask questions about indexed documents using RAG.

Example conceptual request:

```
{
  "question": "Who are the parties to this agreement?",
  "document_ids": [
    "document-id"
  ]
}
```

11.4 Extraction

```
POST /documents/extract
```

Purpose:

Extract structured information from a document.

11.5 Comparison

```
POST /documents/compare
```

Purpose:

Compare two documents and identify differences.

11.6 Agent

```
POST /agent/query
```

Example:

```
{
  "question": "Find the effective date of this agreement and identify the
  parties involved.",
  "document_ids": [
    "e1e91629-c365-41e6-897c-48ec811cc681"
  ]
}
```

The `document_ids` field identifies the indexed document the agent should work with.

12. RAG Architecture

The RAG implementation consists of five major stages.

Stage 1 — Ingestion

The original document is loaded.

```
PDF/Image
↓
ProcessedDocument
```

Stage 2 — Chunking

The document is split into smaller retrieval units.

```
ProcessedDocument
↓
RecursiveCharacterTextSplitter
↓
LangChain Documents
```

Each chunk contains metadata.

Stage 3 — Embedding

Each chunk is converted to a vector using Gemini.

```
Text chunk
↓
Gemini embedding model
↓
Vector
```

Stage 4 — Retrieval

The user question is embedded and compared against FAISS vectors.

```
Question
↓
```

```
Query embedding
↓
FAISS
↓
Relevant chunks
```

Stage 5 — Generation

Retrieved chunks are placed into a grounded prompt.

```
Question + Retrieved Context
↓
Gemini LLM
↓
Grounded Answer
```

13. Structured Extraction

Structured extraction addresses a limitation of normal RAG.

A normal RAG response may be:

```
The agreement became effective on...
```

Structured extraction instead produces machine-readable data.

Conceptually:

```
{
  "field_name": "effective_date",
  "value": "...",
  "confidence": 0.95,
  "source": ...
}
```

This makes extracted information easier to:

- Store
- Validate
- Compare
- Process programmatically
- Display in UI

Pydantic schemas provide validation.

14. Tool-Calling Agent

Why use an agent?

A normal RAG pipeline follows a relatively fixed flow:

```
Question
→ Retrieve
→ Generate
```

The agent adds tool selection:

```
Question
  ↓
Agent
  ↓
Choose appropriate tool
  ↓
Execute tool
  ↓
Observe result
  ↓
Final response
```

This demonstrates an important GenAI engineering concept:

An LLM can act as a decision-making layer over controlled application capabilities.

Available Document Capabilities

The agent has access to document-related tools implemented in:

```
app/agents/tools.py
```

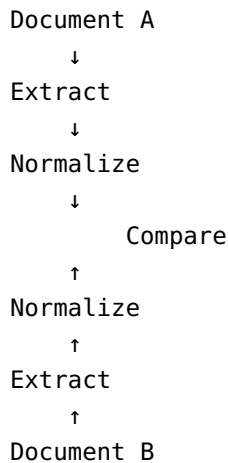
These tools connect the agent with:

- RAG
 - Structured extraction
-

15. Multi-Document Comparison

Comparison was deliberately implemented as a service rather than relying entirely on an LLM to produce a free-form comparison.

The general workflow is:



This allows deterministic Python logic to participate in the final comparison.

This is useful because purely LLM-generated comparisons can be difficult to validate consistently.

16. Testing

The project contains tests for major layers.

Run:

```
pytest -v
```

Test Areas

```
tests/
├─ test_api.py
├─ test_rag.py
├─ test_services.py
├─ test_document.py
├─ test_agent.py
└─ test_evaluator.py
```

Testing philosophy

The project separates:

- Document processing tests
- RAG tests
- Service tests
- Agent tests
- API tests
- Evaluation tests

This makes failures easier to localize.

17. Evaluation Framework

The project contains a dedicated evaluation layer.

Evaluation Questions

Stored in:

```
evaluation/questions.json
```

Questions cover:

- RAG
- Citation
- Extraction
- Reasoning
- Missing information
- Comparison
- Cross-document questions

Evaluation Engine

Implemented in:

```
evaluation/evaluator.py
```

It evaluates responses using deterministic checks.

Citation Evaluation

Citation scoring was specifically designed to consider questions that require citations.

The evaluation framework checks whether:

`citation_found`

and:

`citation_valid`

are satisfied.

This prevents the metric from accidentally ignoring citation-required questions.

18. Important Design Decisions

18.1 FAISS Instead of a Hosted Vector Database

FAISS was selected because the goal was to build a complete working project quickly without introducing unnecessary infrastructure.

Advantages:

- Easy local setup
 - Fast similarity search
 - Simple integration
 - No external database required
-

18.2 Page-Aware Chunking

Documents are not treated as one giant string.

Page metadata is retained because:

Answer
↓
Source
↓
Page

is important for document intelligence.

18.3 Centralized Configuration

Configuration was centralized instead of writing:

```
os.getenv(...)
```

throughout the codebase.

This became especially useful during the OpenAI → Gemini migration.

18.4 Gemini Instead of OpenAI

The initial implementation used OpenAI.

The project was later migrated to Gemini because the OpenAI API payment/card setup was not available.

The migration involved:

- LLM configuration
- Embeddings
- Environment variables
- Dependencies
- Provider validation
- Tests
- Runtime configuration

The final application was successfully tested with Gemini.

18.5 Deterministic Comparison Logic

The comparison service combines LLM extraction with Python comparison/normalization.

This is preferable to asking an LLM to arbitrarily compare two long documents and trusting the result entirely.

18.6 Local Vector Store

FAISS keeps the project simple and makes local development easy.

For a production system, a managed/vector database would be more appropriate.

19. Edge Cases & Error Handling

19.1 Missing API Key

Gemini configuration validates the API key.

A missing key should fail clearly rather than silently producing incorrect behavior.

19.2 Invalid Embedding Model

An important real-world issue was discovered during development.

The initial configuration:

```
text-embedding-004
```

produced:

```
404 NOT_FOUND
```

The actual available models were queried from Gemini.

The working model selected was:

```
gemini-embedding-001
```

This demonstrates why model names should not simply be assumed.

19.3 Empty Document Content

Document processing and downstream components validate/handle missing content where applicable.

19.4 OCR Failures

OCR depends on the local Tesseract installation.

Therefore the application must distinguish between:

```
Python package installed
```

and:

```
Tesseract executable installed
```

19.5 Retrieval Scores

FAISS L2 scores are distances.

They must not be interpreted directly as:

```
confidence = score
```

Lower distance represents stronger similarity.

19.6 Citation Deduplication

The RAG pipeline removes duplicate citations where the same source information is encountered multiple times.

19.7 Agent Iteration Limit

The document agent uses a bounded iteration count to prevent uncontrolled tool-calling loops.

19.8 Document ID Filtering

Retrieval can restrict search to specific document IDs.

This is important when multiple documents exist in the same FAISS store.

19.9 Confidence Validation

Structured extraction confidence values should remain between:

```
0.0
```

and:

1.0

20. Gemini Migration

The project originally used OpenAI-based components.

The migration path was:

```
OPENAI
├──
├── ChatOpenAI
├── OpenAIEmbeddings
└── OPENAI_API_KEY
```

to:

```
GEMINI
├──
├── Gemini LLM
├── Gemini Embeddings
└── GEMINI_API_KEY
```

The migration required changes across:

- Configuration
- Embeddings
- RAG pipeline
- Extraction service
- Agent
- Dependencies
- Tests

Final Embedding Configuration

The working embedding model was verified dynamically through the Gemini API.

Available models returned:

```
models/gemini-embedding-001
models/gemini-embedding-2-preview
models/gemini-embedding-2
```

The stable selected model was:

```
gemini-embedding-001
```

Verification

The final system was tested through:

```
POST /documents/upload
```

and document indexing succeeded.

The RAG `/chat` route was also successfully tested.

The Streamlit application was subsequently tested end-to-end and the project functionality was confirmed working.

21. Known Limitations

Local FAISS

The current architecture is primarily designed for local development.

A production deployment would likely require:

- Managed vector storage
 - Authentication
 - Multi-user isolation
 - Persistent object storage
 - Distributed processing
-

OCR Accuracy

OCR quality depends on:

- Image resolution
 - Scan quality
 - Font
 - Document layout
 - Tesseract configuration
-

LLM Reliability

Even with retrieval grounding, LLM-generated answers should be treated as AI-generated information.

For legal or business-critical documents, the original source should remain the final authority.

Comparison Normalization

Generic normalization can create false equivalence in cases where formatting differences are semantically important.

For example:

00123

and:

123

may not always represent the same identifier.

22. Future Improvements

The project deliberately avoided adding unnecessary infrastructure during the initial one-week build.

Potential future improvements include:

22.1 PostgreSQL + pgvector

Move from local FAISS to a persistent production-oriented vector database.

22.2 LangGraph

Introduce explicit graph-based agent workflows for complex multi-step reasoning.

22.3 Advanced Evaluation

Add:

- Ragas
- DeepEval

- More comprehensive retrieval metrics
 - LLM-as-judge evaluation
-

22.4 Authentication

Add:

- User accounts
 - Authentication
 - Authorization
 - Document ownership
-

22.5 Cloud Storage

Store uploaded documents in:

- S3
- Google Cloud Storage
- Azure Blob Storage

rather than local disk.

22.6 Background Processing

Large documents could be processed asynchronously through:

```
graph TD;
  Upload --> JobQueue[Job Queue];
  JobQueue --> Worker;
  Worker --> Embedding;
  Embedding --> Indexing;
```

22.7 Hybrid Search

Combine:

```
Vector Search
+
Keyword/BM25 Search
```

for better retrieval on identifiers, names and exact terminology.

22.8 Document Versioning

Add version tracking so users can compare:

```
Contract v1
Contract v2
Contract v3
```

and visualize changes over time.

22.9 Observability

Future production deployment could include:

- Request tracing
 - LLM latency
 - Token usage
 - Retrieval metrics
 - Error tracking
 - Agent tool-call monitoring
-

23. Project Revision Checklist

When revisiting this project later, remember the following sequence.

Step 1 — Start Backend

```
uvicorn app.api.main:app --reload
```

Step 2 — Open Swagger

```
http://127.0.0.1:8000/docs
```

Step 3 — Start Frontend

```
streamlit run frontend/app.py
```

Step 4 — Verify Configuration

Check:

```
GEMINI_API_KEY  
LLM_PROVIDER  
LLM_MODEL  
EMBEDDING_PROVIDER  
EMBEDDING_MODEL
```

Step 5 — Test Upload

```
POST /documents/upload
```

If upload fails during embedding, inspect:

```
app/rag/embeddings.py  
app/core/config.py
```

and verify the Gemini embedding model.

Step 6 — Test RAG

Use:

```
POST /chat
```

Verify:

- Answer quality
 - Retrieval
 - Citations
 - Correct document filtering
-

Step 7 — Test Extraction

Use:

```
POST /documents/extract
```

Verify:

- Fields
 - Confidence
 - Sources
 - Structured output
-

Step 8 — Test Agent

Use:

```
POST /agent/query
```

Example:

```
Find the effective date of this agreement and identify the parties involved.
```

Step 9 — Test Comparison

Use:

```
POST /documents/compare
```

Verify:

- Matching fields
 - Normalization
 - Differences
 - Mismatch summary
-

Step 10 — Run Tests

```
pytest -v
```

24. Resume-Level Project Summary

Short Description

DocuMind AI — Intelligent Document Analysis & Agent Platform

Built a GenAI-powered document intelligence platform using **Python, LangChain, Gemini, FAISS, OCR, FastAPI and Streamlit**, supporting document ingestion, page-aware RAG, source citations, structured information extraction, tool-calling agents and multi-document comparison.

Resume Bullets

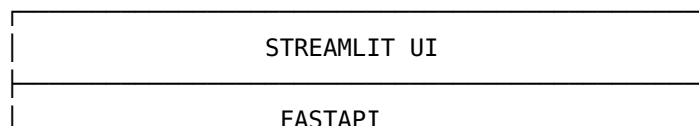
- Built a document intelligence platform using **LangChain, Gemini and FAISS**, implementing page-aware Retrieval-Augmented Generation with source/document citations for grounded question answering.
- Developed structured document extraction and **tool-calling agent workflows** using Pydantic schemas and controlled document tools for information retrieval and extraction.
- Implemented **multi-document comparison and inconsistency detection**, along with PDF/OCR processing, FastAPI REST APIs, Streamlit UI, automated tests and a dedicated evaluation framework.

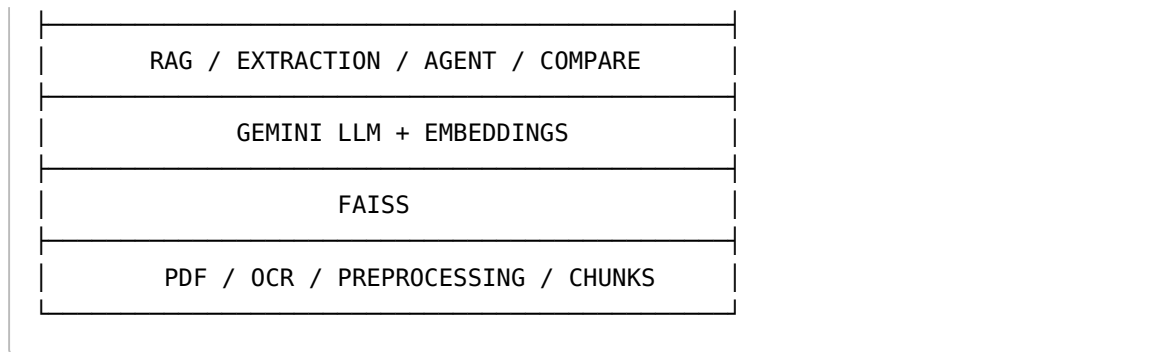
Technology Stack

```
Python
LangChain
Gemini
FAISS
RAG
OCR
PyMuPDF
Pydantic
FastAPI
Streamlit
pytest
Docker
```

Final Project Mental Model

When revising the project, remember it as six layers:

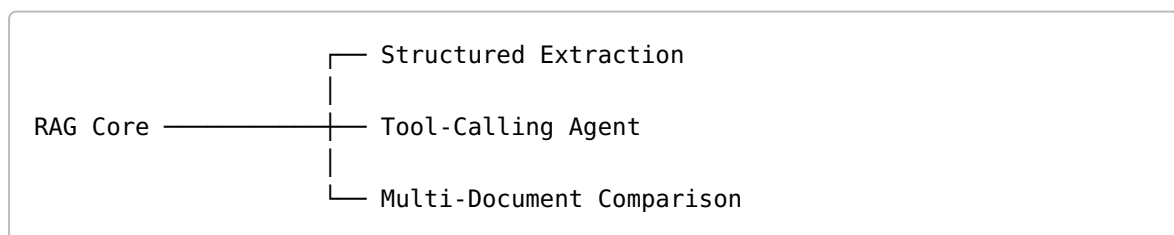




The most important conceptual flow to remember is:



And around that core RAG pipeline, DocuMind adds:



That is the central architecture and mental model to retain when returning to the project.